

SRS — Software Requirements Specification

Dự án: Hệ thống Đặt vé xe khách trực tuyến "Pam Travel"

Thông tin tài liệu

Thông tin	Chi tiết
Dự án	Pam Travel
Phiên bản	1.2
Ngày cập nhật	05/06/2026
Nhóm thực hiện	Dev Team
Công nghệ	Java 21 · Spring Boot 3.x · Vue.js 3 · PostgreSQL 18

1. Tổng quan hệ thống

Pam Travel là hệ thống đặt vé xe khách trực tuyến cho phép hành khách tìm kiếm chuyến đi, chọn ghế theo thời gian thực, thanh toán trực tuyến và sử dụng vé điện tử QR Code.

Thành phần hệ thống:

- Web Frontend — Vue.js 3 + Vite + TailwindCSS
- Backend API — Spring Boot 3.x
- Database — PostgreSQL 18
- Payment Gateway — VNPay, MoMo
- QR Ticket Service

1.1 Vai trò người dùng

CUSTOMER

- Đăng ký / Đăng nhập
- Tìm kiếm chuyến xe
- Chọn ghế trên sơ đồ động
- Đặt vé và thanh toán trực tuyến
- Xem vé điện tử (QR Code)
- Hủy vé
- Đánh giá chuyến đi

DRIVER

- Đăng nhập

- Xem danh sách hành khách
- Quét QR Check-in
- Xác nhận hành khách lên xe

ADMIN

- Quản lý người dùng
- Quản lý nhà xe và tuyến đường
- Quản lý chuyến xe
- Quản lý thanh toán và hoàn tiền
- Xem báo cáo / Dashboard

2. Yêu cầu chức năng (Functional Requirements)

REQ-01: Authentication

Mô tả: Xác thực người dùng qua Email hoặc Số điện thoại, trả về JWT Token.

Công nghệ: Spring Security · JWT · BCrypt Password Hashing

Quy trình:

1. Người dùng gửi thông tin đăng nhập (email/phone + password).
2. Spring Security xác thực qua `AuthenticationManager` + `password_hash`.
3. Nếu hợp lệ, sinh JWT Token trả về Client.
4. Token được đính kèm trong `Authorization: Bearer <token>` cho các request tiếp theo.

Endpoints:

```
POST /api/v1/auth/register
POST /api/v1/auth/login
```

REQ-02: Trip Search

Mô tả: Tìm kiếm chuyến xe theo tuyến đường và ngày khởi hành.

Đầu vào:

Tham số	Kiểu	Bắt buộc
<code>originId</code>	Long	✓
<code>destinationId</code>	Long	✓
<code>departureDate</code>	Date	✓

Xử lý: Query JOIN bảng `trips`, `routes`, `route_stops`, lọc theo tuyến đường, ngày khởi hành và `status = 'OPEN'`.

Đầu ra: Danh sách chuyến xe gồm giờ đi, giờ đến, thời gian di chuyển, giá vé, loại xe, số ghế còn trống.

Endpoint:

```
GET /api/v1/trips/search?originId=1&destinationId=3&departureDate=2026-06-10
```

REQ-03: Dynamic Seat Map

Mô tả: Trả về cấu hình sơ đồ ghế và trạng thái từng ghế theo thời gian thực. Frontend Vue.js render động dựa trên JSON này.

Endpoint:

```
GET /api/v1/trips/{tripId}/seat-map
```

Response mẫu:

```
{
  "tripId": 45,
  "vehicleType": "Limousine 34",
  "layout": {
    "floors": 2,
    "floor_1": { "rows": 6, "cols": 3, "aisle_after_col": 1 },
    "floor_2": { "rows": 6, "cols": 3, "aisle_after_col": 1 }
  },
  "seats": [
    { "seatId": 12, "seatName": "1-A1", "status": "CONFIRMED" },
    { "seatId": 15, "seatName": "1-A2", "status": "LOCKED" },
    { "seatId": 16, "seatName": "1-A3", "status": "AVAILABLE" }
  ]
}
```

Trạng thái ghế:

Trạng thái	Mô tả
AVAILABLE	Ghế trống, có thể chọn
LOCKED	Đang được giữ tạm (tối đa 10 phút)
CONFIRMED	Đã thanh toán thành công

REQ-04: Booking & Seat Locking

Mô tả: Giữ chỗ tạm thời trước khi thanh toán, tránh double-booking.

Endpoint:

```
POST /api/v1/bookings
```

Request:

```
{
  "tripId": 45,
  "pickupStopId": 12,
  "dropoffStopId": 19,
  "selectedSeatIds": [12, 13]
}
```

Quy trình xử lý:

1. Kiểm tra chuyến xe tồn tại và đang **OPEN**.
2. Kiểm tra các ghế thuộc đúng chuyến xe.
3. Mở Database Transaction.
4. **SELECT FOR UPDATE** trên các hàng ghế — ngăn race condition.
5. Kiểm tra tất cả ghế đang **AVAILABLE**.
6. Cập nhật trạng thái ghế → **LOCKED**, `locked_until = NOW() + 10 phút`.
7. Tạo Booking trạng thái **PENDING**, `expires_at = NOW() + 10 phút`.
8. Commit Transaction, trả về response.

Business Rules:

- Tối đa 5 ghế mỗi giao dịch.
- Ghế được giữ tối đa 10 phút — hết hạn tự động về **AVAILABLE**.
- Nếu 2 request cùng lúc chọn 1 ghế: request đến trước thắng, request sau nhận **409 Conflict**.

Response thành công (201):

```
{
  "bookingId": 101,
  "status": "PENDING",
  "expiresAt": "2026-06-05T12:10:00Z"
}
```

Response lỗi (409):

```
{
  "error": "SEAT_ALREADY_LOCKED",
  "message": "Ghế vừa được người khác đặt"
}
```

REQ-05: Payment Processing

Mô tả: Tích hợp cổng thanh toán trực tuyến, xác thực webhook callback.

Cổng thanh toán hỗ trợ: VNPay · MoMo

Endpoints:

```
POST /api/v1/payments/create
POST /api/v1/payments/webhook
```

Trạng thái thanh toán:

Trạng thái	Mô tả
PENDING	Chờ người dùng thanh toán
SUCCESS	Thanh toán thành công
FAILED	Thanh toán thất bại
REFUNDED	Đã hoàn tiền

Luồng xử lý:

```
Booking (PENDING)
  → Tạo Payment (PENDING)
  → Chuyển hướng sang cổng thanh toán
  → Người dùng thanh toán
  → Payment Webhook Callback
  → Xác thực Signature (bắt buộc)
  → Cập nhật Payment → SUCCESS
  → Booking → CONFIRMED
  → Sinh Ticket + QR Code
```

Yêu cầu bảo mật: Phải xác thực chữ ký (Signature) từ cổng thanh toán. Không cập nhật trạng thái nếu Signature không hợp lệ.

REQ-06: Electronic Ticket (E-Ticket)

Mô tả: Phát hành vé điện tử sau khi thanh toán thành công. Mỗi ghế tương ứng một vé độc lập.

Endpoint:

```
GET /api/v1/tickets/{ticketId}
```

Thông tin vé:

- Ticket ID
- Booking ID / Trip ID
- Tên hành khách
- Số ghế
- Tuyến đường + giờ khởi hành
- QR Code (mã hóa bảo mật)

Trạng thái vé:

Trạng thái	Mô tả
ISSUED	Vé hợp lệ, chưa dùng
USED	Đã check-in lên xe
CANCELLED	Vé đã hủy

Quy tắc:

- Vé chỉ được tạo khi `Payment.status = SUCCESS`.
- QR Code phải chứa dữ liệu xác thực từ hệ thống (không thể làm giả).

REQ-07: QR Check-in

Mô tả: Tài xế quét mã QR để xác nhận hành khách lên xe.

Endpoint:

```
POST /api/v1/tickets/check-in
```

Quy trình:

1. Quét QR → giải mã dữ liệu vé.
2. Kiểm tra vé tồn tại trong hệ thống.
3. Kiểm tra trạng thái vé là `ISSUED`.
4. Cập nhật trạng thái → `USED`, ghi `checked_in_at` và `checked_in_by`.
5. Trả về thông tin hành khách và ghế.

Các lỗi có thể xảy ra:

Mã lỗi	Mô tả
INVALID_QR	QR không hợp lệ
TICKET_NOT_FOUND	Không tìm thấy vé
TICKET_ALREADY_USED	Vé đã được sử dụng
TICKET_CANCELLED	Vé đã bị hủy

REQ-08: Notification Management

Mô tả: Quản lý thông báo trong hệ thống.

Endpoint:

```
GET /api/v1/notifications
PATCH /api/v1/notifications/{id}/read
```

Loại thông báo:

Loại	Trigger
BOOKING_CREATED	Đặt vé thành công
PAYMENT_SUCCESS	Thanh toán thành công
PAYMENT_FAILED	Thanh toán thất bại
TRIP_REMINDER	Nhắc lịch trước 2 giờ khởi hành
REFUND_SUCCESS	Hoàn tiền hoàn tất
SYSTEM_NOTIFICATION	Thông báo hệ thống

REQ-09: Authorization (RBAC)

Mô tả: Phân quyền theo Role-Based Access Control.

Ma trận phân quyền:

Chức năng	CUSTOMER	DRIVER	ADMIN
Đặt vé	✓	X	X
Xem vé cá nhân	✓	X	✓
Hủy vé	✓	X	✓
Check-in QR	X	✓	✓
Xem danh sách khách	X	✓	✓

Chức năng	CUSTOMER	DRIVER	ADMIN
Quản lý chuyến xe	X	X	✓
Quản lý người dùng	X	X	✓
Xem Dashboard / Report	X	X	✓

REQ-10: Real-time Seat Status

Mô tả: Đồng bộ trạng thái ghế tức thời giữa nhiều người dùng đang xem cùng chuyến xe.

Giải pháp:

- Ưu tiên: **WebSocket** (STOMP over SockJS)
- Dự phòng: **Polling mỗi 5 giây**

Event mẫu:

```
{
  "tripId": 45,
  "seatId": 12,
  "status": "LOCKED"
}
```

3. Danh sách API Endpoints

Method	Endpoint	Role	Mô tả
POST	/api/v1/auth/register	Public	Đăng ký
POST	/api/v1/auth/login	Public	Đăng nhập
GET	/api/v1/trips/search	Public	Tìm kiếm chuyến xe
GET	/api/v1/trips/{id}/seat-map	Public	Lấy sơ đồ ghế
POST	/api/v1/bookings	CUSTOMER	Tạo booking
GET	/api/v1/bookings/{id}	CUSTOMER / ADMIN	Chi tiết booking
POST	/api/v1/payments/create	CUSTOMER	Khởi tạo thanh toán
POST	/api/v1/payments/webhook	System	Callback cổng TT
GET	/api/v1/tickets/{id}	CUSTOMER / ADMIN	Lấy vé
POST	/api/v1/tickets/check-in	DRIVER / ADMIN	Soát vé QR
GET	/api/v1/notifications	CUSTOMER	Danh sách thông báo
PATCH	/api/v1/notifications/{id}/read	CUSTOMER	Đánh dấu đã đọc

4. Ràng buộc kỹ thuật (Technical Constraints)

Backend

- Java **21 LTS**
- Spring Boot **3.x**
- Spring Data JPA + Hibernate
- Spring Security + JWT

Frontend

- Vue.js **3** (Composition API)
- Vite
- TailwindCSS — render sơ đồ ghế động từ JSON, không hard-code CSS

Database

- PostgreSQL **18**
- Kiểu **JSONB** cho **seat_layout_json**
- **Partial Index** cho seat locking
- **TIMESTAMPTZ** cho tất cả timestamp

Bảo mật

- JWT Authentication
- BCrypt Password Hashing
- HTTPS bắt buộc
- Xác thực Payment Signature

5. Database Design

Danh sách 17 bảng

Nhóm	Bảng
Master Data	provinces , operators , vehicle_types
Xe & Ghế	vehicles , seats
Tuyến đường	routes , route_stops
Chuyến đi	trips , trip_seat_status
Người dùng	users
Đặt vé	bookings , booking_seats
Thanh toán	payments , tickets , refunds
Phụ trợ	reviews , notifications

Index đã tạo

```
-- Tìm kiếm chuyến xe nhanh
CREATE INDEX idx_trips_route_departure
  ON trips(route_id, departure_time);

-- Seat locking – chỉ index ghế đang bị khóa
CREATE INDEX idx_tss_locked
  ON trip_seat_status(trip_id, locked_until)
  WHERE status = 'LOCKED';

-- My Tickets của khách
CREATE INDEX idx_bookings_user ON bookings(user_id);

-- Dashboard nhà xe
CREATE INDEX idx_bookings_trip ON bookings(trip_id);

-- Thông báo chưa đọc
CREATE INDEX idx_notifications_unread
  ON notifications(user_id)
  WHERE is_read = FALSE;
```

6. Non-Functional Requirements

Mã	Yêu cầu	Mục tiêu
NFR-01	Performance	Search API < 2s · Seat Map API < 1s · Booking API < 2s
NFR-02	Scalability	Tối thiểu 1.000 người dùng đồng thời · Có thể scale nhiều instances
NFR-03	Availability	Uptime ≥ 99%
NFR-04	Security	JWT · BCrypt · HTTPS · Verify Payment Signature
NFR-05	Reliability	Không xảy ra Double Booking · Toàn bộ Booking chạy trong Transaction
NFR-06	Logging & Audit	Ghi log: Login · Booking · Payment · Refund · Check-in (User ID, Timestamp, Action, IP)

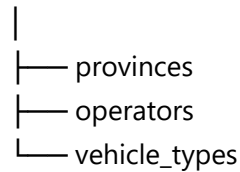
7. Lộ trình phát triển

Giai đoạn	Nội dung	Trạng thái
-----------	----------	------------

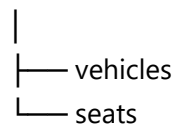
Giai đoạn	Nội dung	Trạng thái
MVP	Auth · Trip Search · Seat Map · Booking · Payment · QR Ticket	 Đang làm
Phase 2	Hủy vé tự động · Đối soát dòng tiền · Zalo ZNS	☐ Chưa làm
Phase 3	Tích điểm · Đánh giá · Push Notification	☐ Chưa làm
Phase 4	AI đề xuất tuyến · Live Tracking GPS	☐ Chưa làm

Bức tranh tổng thể hệ thống

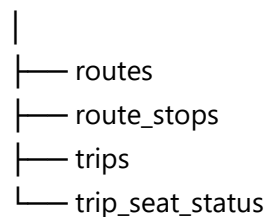
MASTER DATA



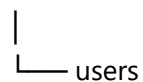
XE



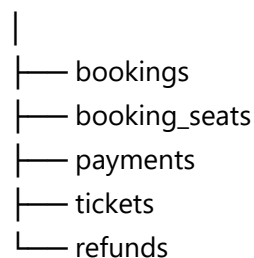
TUYẾN & CHUYẾN



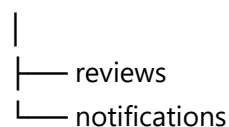
NGƯỜI DÙNG



ĐẶT VÉ



PHỤ TRỢ



Ghi chú

Tài liệu này là bản đặc tả yêu cầu phần mềm cho phiên bản MVP của Pam Travel. Các tính năng Loyalty Program, Voucher, Dynamic Pricing và Live Tracking sẽ được đặc tả ở các phiên bản tiếp theo.